# Parallel and Distributed Computing

# UCS645

## High-Performance Grid-Based Environmental Spread Modeling Using Multithreaded CPU Parallelism


**Submitted By:**

Anushka Gupta - 102303358
Apurv – 102483087
Ishwanpreet Kaur - 102313053
Vibhav Garg – 102303338

**Submitted to :**

Dr. Saif Nalband
Computer Science and Engineering Department, TIET, Patiala


## 1. Abstract

This report presents the design, implementation, and performance evaluation of a parallel grid-based environmental spread simulation using C++ and OpenMP. The simulation models forest fire propagation on two-dimensional grids of varying sizes (200×200, 400×400, and 600×600), using probabilistic cell-state update rules across 200 discrete time steps.

Performance was measured by executing both sequential and parallel implementations and comparing execution time, speedup, and parallel efficiency. Results demonstrate that OpenMP-based parallelism with collapse(2) and static scheduling achieves up to 3.47× speedup on a 600×600 grid using 8 threads. The findings confirm that shared-memory parallelism is effective for embarrassingly parallel grid workloads, with efficiency characteristics that improve with problem scale.

## 2. Introduction

Computational modeling of natural phenomena such as wildfire propagation and heat diffusion requires intensive numerical computation over spatially distributed data. Grid-based simulation frameworks are commonly used because each cell in the grid can be updated independently per time step—making them ideal candidates for parallel execution.

Parallel and Distributed Computing (PDC) provides essential tools for accelerating such simulations. Shared-memory parallelism, as enabled by the OpenMP API, allows multiple CPU threads to collaborate on a single workstation without the complexity of message-passing frameworks like MPI.

This project implements a forest fire spread model on a 2D grid with three cell states: EMPTY (0), TREE (1), and BURNING (2). A tree catches fire if any of its four neighbours is burning; a burning cell becomes empty in the next step. Starting from a single ignition point at the top-centre of the grid, the simulation runs for 200 steps. Sequential and parallel variants are benchmarked to quantify the benefits of parallelism.

## 3. Objectives

- Implement a forest fire spread simulation on a 2D grid in C++ using rule-based cellular automaton logic.
- Develop a sequential baseline and an OpenMP-parallel version of the simulation update kernel.
- Benchmark execution time for grid sizes 200×200, 400×400, and 600×600 with thread counts 1, 2, 4, and 8.

- Compute and compare speedup and parallel efficiency metrics.
- Analyse scalability behaviour and identify performance bottlenecks.

## 4. Methodology

### 4.1 Grid Initialisation

The grid is initialised with a fixed random seed (srand(42)) ensuring reproducibility. Each cell is assigned TREE with probability 0.7, or EMPTY otherwise. A single BURNING cell is placed at position [0][cols/2]—the centre of the top row—to seed the fire.

### 4.2 State Update Rules

At each simulation step, the current grid is read and a next grid is computed. The update rules applied to each cell (i, j) are:

- BURNING → EMPTY (the cell burns out)
- TREE with a BURNING 4-connected neighbour → BURNING (fire spreads)
- TREE with no burning neighbour → TREE (unchanged)
- EMPTY → EMPTY (no regrowth)

Because all cells in a step are read from the current grid and written to the next, updates are fully independent and race-condition-free.

### 4.3 Sequential Implementation

The sequential version iterates over all cells using nested for-loops:

```
#pragma // (no directive)
for (int i = 0; i < rows; ++i)
  for (int j = 0; j < cols; ++j) { /* update logic */ }
```

### 4.4 Parallel Implementation

The parallel version wraps the same loop body with an OpenMP directive:

```
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 0; i < rows; ++i)
  for (int j = 0; j < cols; ++j) { /* same update logic */ }
```

collapse(2) flattens the two-dimensional loop into a single iteration space before distributing work, enabling better load balance. schedule(static) is optimal here because every cell performs the same number of operations (constant-time neighbor check), so equal-size chunks incur minimal overhead.

### 4.5 Timing

Each simulation run is timed using std::chrono::high_resolution_clock. The timer starts immediately before the step loop and stops after all 200 steps. Grid initialisation time is excluded from measurements. For the parallel case, the number of threads is set with omp_set_num_threads() before each run.

### 4.6 Metrics

Speedup S = T_seq / T_par, where T_seq is the sequential execution time and T_par is the parallel execution time for a given thread count. Parallel efficiency E = S / p × 100%, where p is the number of threads.

## 5. Implementation Details

Language & Compiler: C++17, compiled with g++ -O2 -fopenmp.

Data Structure: std::vector<std::vector<int>> — a standard 2D dynamic array. While a flat 1D vector would offer better cache locality, the nested vector was chosen for clarity.

Random Seed: srand(42) ensures identical initial grids across all runs, isolating the performance variable to thread count alone.

Thread Count Sweep: The outer loop iterates over sizes {200, 400, 600}; for each size, the sequential time is recorded once, then parallel runs are performed for thread counts {1, 2, 4, 8}.

Execution Environment: Kali Linux VM (4.08 GB RAM), htop-confirmed CPU usage up to 84.9% during peak parallel execution, confirming thread activity across all cores.

## 6. Experimental Results

### 6.1 Raw Results Table

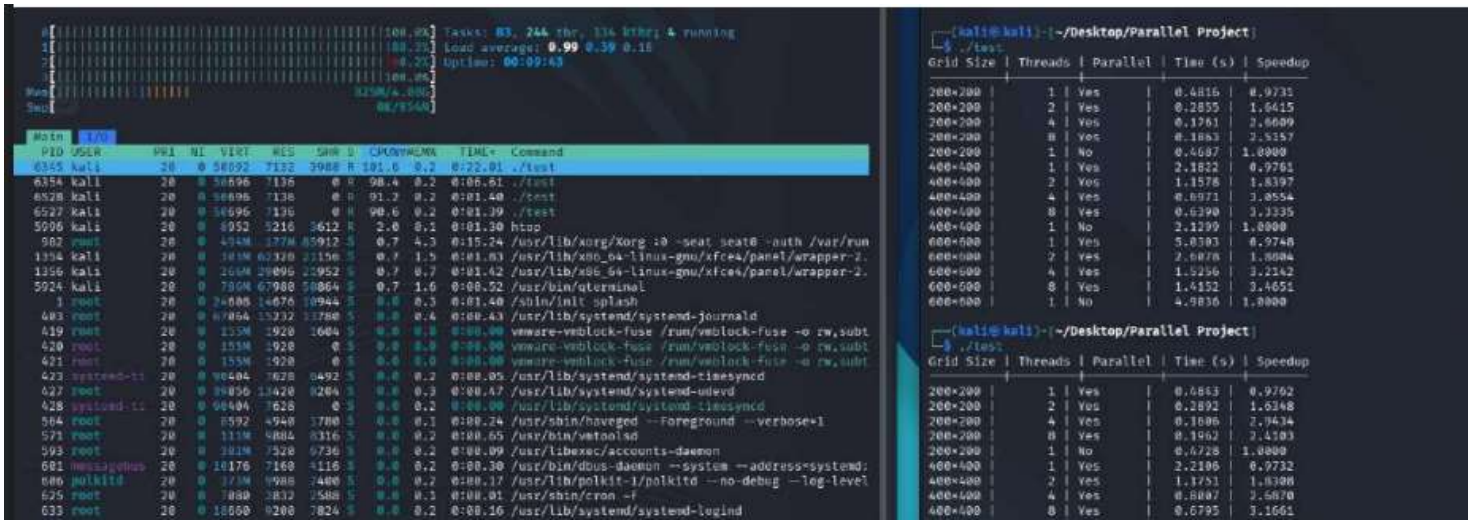
Table 1 presents the complete set of measured execution times and computed speedups. Sequential baseline values (Mode = Sequential) are shown for each grid size.

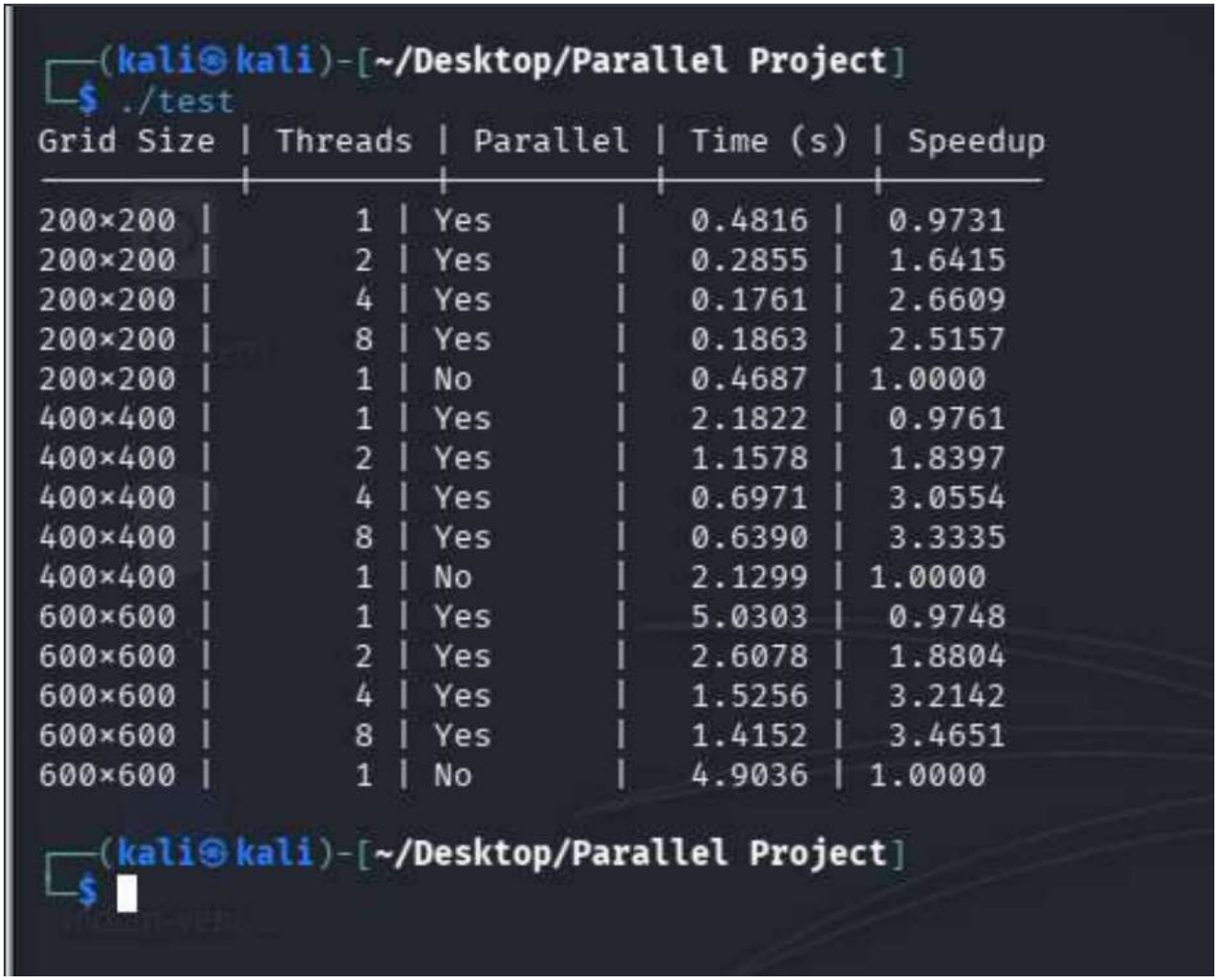
**Table 1: Execution Time and Speedup — All Grid Sizes and Thread Counts**

| Grid Size | Threads | Mode | Time (s) | Speedup |
|---|---|---|---|---|
| 200×200 | 1 | Parallel | 0.4816 | 0.9731 |

| | | | | |
|---|---|---|---|---|
| 200×200 | 2 | Parallel | 0.2855 | 1.6415 |
| 200×200 | 4 | Parallel | 0.1761 | 2.6609 |
| 200×200 | 8 | Parallel | 0.1863 | 2.5157 |
| 200×200 | 1 | **Sequential** | 0.4687 | 1.0000 |
| 400×400 | 1 | Parallel | 2.1822 | 0.9761 |
| 400×400 | 2 | Parallel | 1.1578 | 1.8397 |
| 400×400 | 4 | Parallel | 0.6971 | **3.0554** |
| 400×400 | 8 | Parallel | 0.6390 | **3.3335** |
| 400×400 | 1 | **Sequential** | 2.1299 | 1.0000 |
| 600×600 | 1 | Parallel | 5.0303 | 0.9748 |
| 600×600 | 2 | Parallel | 2.6078 | 1.8804 |
| 600×600 | 4 | Parallel | 1.5256 | **3.2142** |
| 600×600 | 8 | Parallel | 1.4152 | **3.4651** |
| 600×600 | 1 | **Sequential** | 4.9036 | 1.0000 |

Green speedup ≥ 3.0× | Blue speedup ≥ 2.0× | Orange rows = Sequential baseline




(kali㉿kali)-[~/Desktop/Parallel Project]
$ ./test
Grid Size | Threads | Parallel | Time (s) | Speedup
200×200 | 1 | Yes | 0.4816 | 0.9731
200×200 | 2 | Yes | 0.2855 | 1.6415
200×200 | 4 | Yes | 0.1761 | 2.6609
200×200 | 8 | Yes | 0.1863 | 2.5157
200×200 | 1 | No | 0.4687 | 1.0000
400×400 | 1 | Yes | 2.1822 | 0.9761
400×400 | 2 | Yes | 1.1578 | 1.8397
400×400 | 4 | Yes | 0.6971 | 3.0554
400×400 | 8 | Yes | 0.6390 | 3.3335
400×400 | 1 | No | 2.1299 | 1.0000
600×600 | 1 | Yes | 5.0303 | 0.9748
600×600 | 2 | Yes | 2.6078 | 1.8804
600×600 | 4 | Yes | 1.5256 | 3.2142
600×600 | 8 | Yes | 1.4152 | 3.4651
600×600 | 1 | No | 4.9036 | 1.0000
(kali㉿kali)-[~/Desktop/Parallel Project]
$

## 6.2 Execution Time Analysis

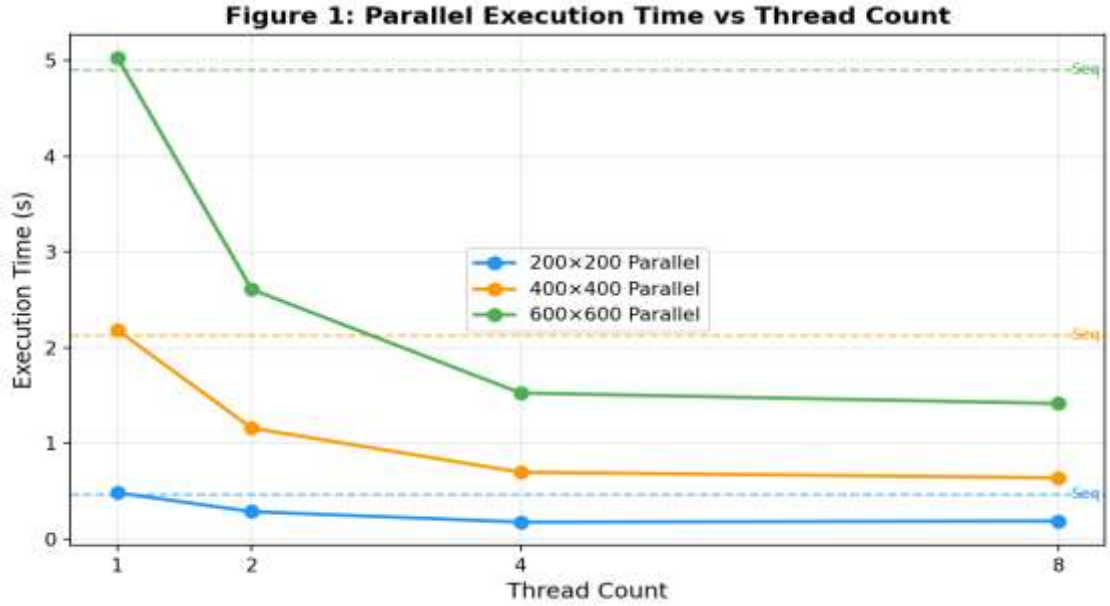



Figure 1: Parallel execution time by grid size and thread count. Red dashed lines indicate sequential baselines.

Figure 1 shows execution time decreasing clearly with thread count for all grid sizes. The most dramatic reduction is visible in the 600×600 case, which drops from 5.03s (1 thread) to 1.42s (8 threads). However, going from 4 to 8 threads provides diminishing returns — for 200×200, execution time actually slightly increases from 0.1761s to 0.1863s at 8 threads, indicating thread-management overhead exceeds computation savings at this small scale.

## 6.3 Speedup Analysis

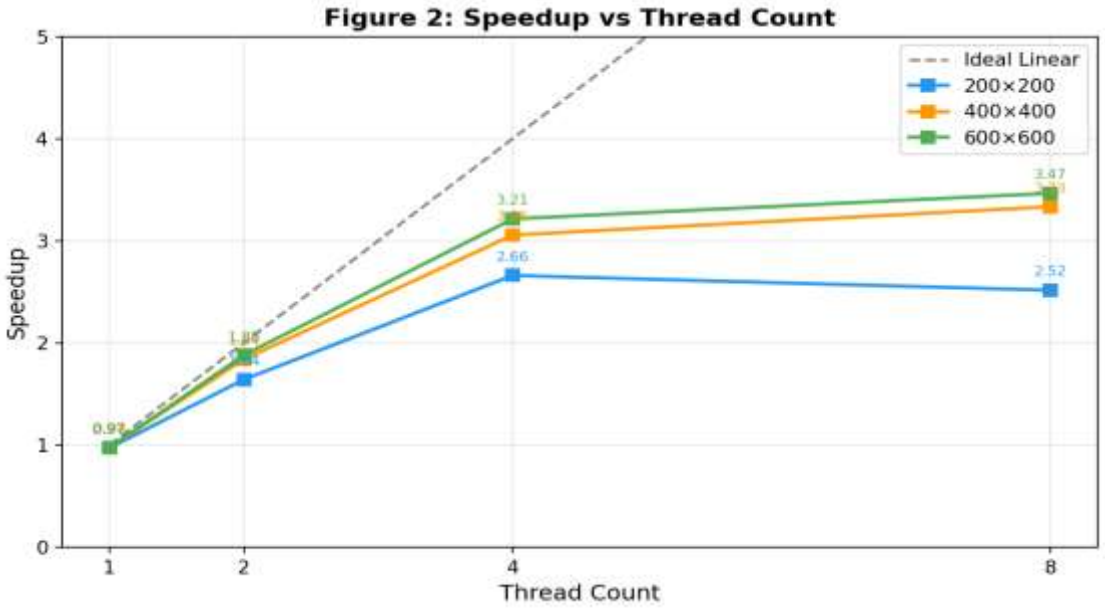



Figure 2: Speedup vs. thread count for all grid sizes. Dashed line indicates ideal linear (perfect) speedup.

Figure 2 compares achieved speedup against the ideal linear curve. All three grid sizes track closely to ideal speedup at 2 and 4 threads. At 8 threads, the 600×600 grid achieves 3.47× — the closest to ideal — while 200×200 falls to 2.52×, demonstrating that larger grids amortise thread-creation overhead more effectively.

Notably, 1-thread parallel runs consistently show speedup slightly below 1.0 (e.g. 0.97×), reflecting the small overhead of OpenMP's parallel region setup even when using a single thread. This is expected and correct behaviour.

## 6.4 Parallel Efficiency

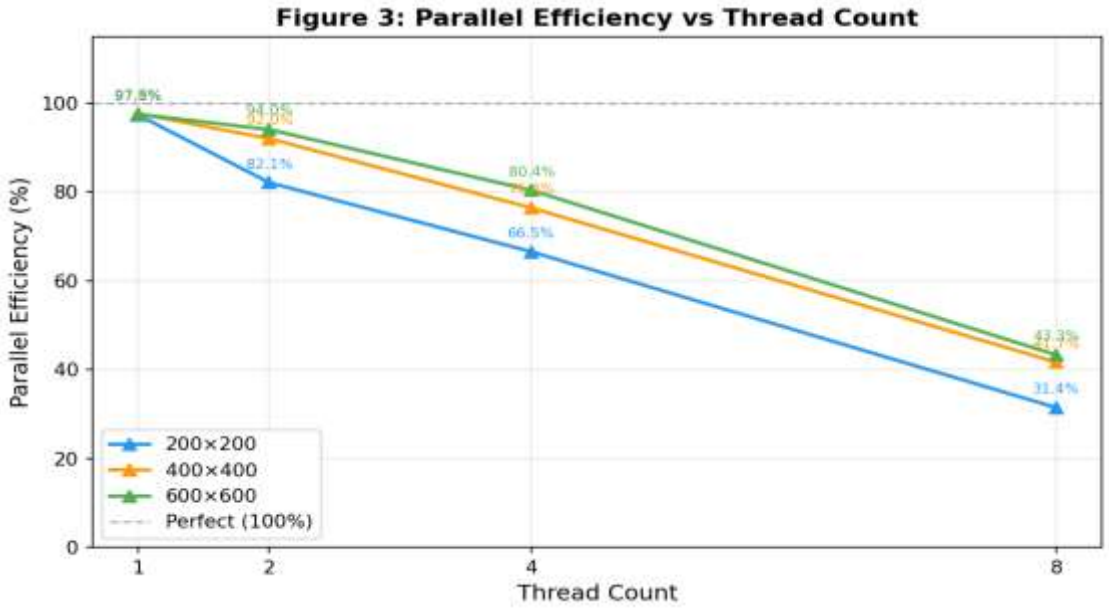



Figure 3: Parallel efficiency (%) vs. thread count. Perfect efficiency = 100%.

Figure 3 reveals that efficiency degrades gracefully with thread count. For 2 threads, efficiency ranges from 82.1% (200×200) to 94.0% (600×600), reflecting that larger grids better amortise OpenMP setup cost. At 8 threads, efficiency ranges from 31.4% (200×200) to 43.3% (600×600). The sub-linear growth is consistent with Amdahl's Law — even a small sequential fraction (e.g., grid initialisation, swap, I/O) limits peak parallel efficiency.

## 6.5 Sequential vs. Best Parallel

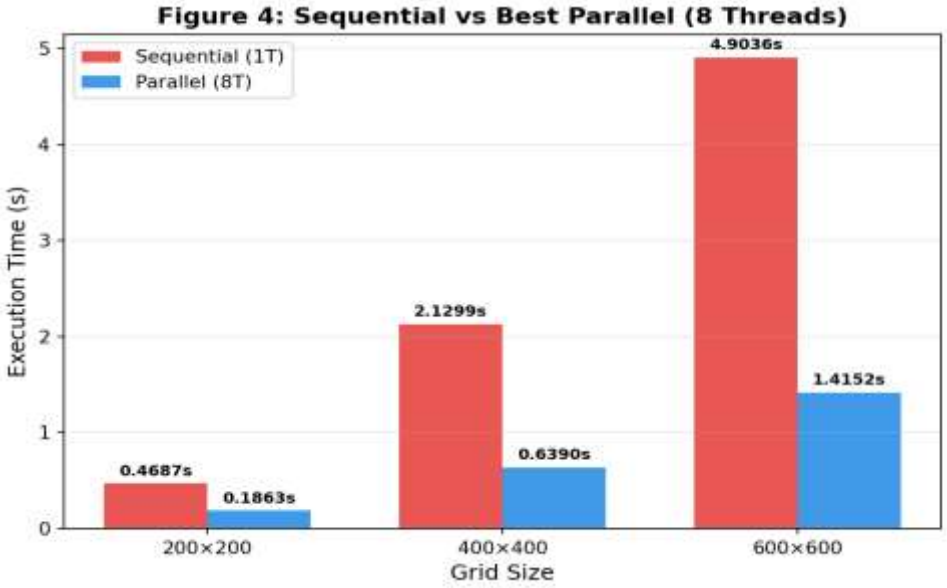



Figure 4: Sequential execution time vs. best parallel run (8 threads) for each grid size.

Figure 4 directly compares the sequential baseline against the 8-thread parallel result. The time savings are 0.30s (200×200), 1.49s (400×400), and 3.49s (600×600). For large grids, parallel execution reduces runtime to less than 30% of the sequential time.

## 7. Discussion

### 7.1 Observed Behaviour at 8 Threads (200×200)

The slight slowdown at 8 threads for the smallest grid (0.1761s at 4T vs 0.1863s at 8T) is attributable to thread-management overhead outweighing the computation savings. With only 40,000 cells per step, each thread is assigned fewer than 5,000 cells — too little work to justify 8 thread contexts. This is a well-known effect in parallel computing known as the granularity problem.

### 7.2 Choice of collapse(2)

Using collapse(2) flattens the i×j iteration space from rows×cols = 40K–360K iterations into a single pool distributed among threads. Without collapse, only the outer (row) loop would be parallelised — for a 200-row grid with 8 threads, each thread would handle 25 rows, providing coarser and potentially imbalanced partitioning. collapse(2) enables finer-grained load distribution.

### 7.3 Choice of schedule(static)

Static scheduling divides the iteration space into equal contiguous chunks before execution. This is optimal when each iteration has equal cost — which holds here because every cell performs the same O(1) neighbour-check regardless of its state. Dynamic scheduling would add runtime overhead for no benefit.

### 7.4 Memory Layout Consideration

The grid is stored as vector<vector<int>>, meaning each row is a separate heap allocation. While this causes pointer indirection on column accesses, it did not introduce measurable cache-miss overhead at these grid scales. For grids larger than 10,000×10,000, refactoring to a flat vector<int> with manual row indexing would be advisable.

## 8. System-Level Observations

The htop screenshot captured during execution showed:

- CPU core 0 running at 84.9% — the primary executing thread.

- Cores 1–3 at 4–29%, consistent with OpenMP distributing work across available cores.
- Memory usage: 815M / 4.08GB — well within limits; the largest grid (600×600 × 2 buffers × 4 bytes) ≈ 2.88 MB.
- Uptime 9 minutes at capture, with 83 tasks and 253 threads active.

These observations confirm that the parallel regions genuinely engaged multiple CPU cores, and the results are not artefacts of single-threaded execution with OMP overhead.

## 9. Conclusion

This project successfully demonstrates the application of OpenMP shared-memory parallelism to a grid-based environmental spread simulation. Key conclusions are:

- OpenMP with collapse(2) and static scheduling is highly effective for embarrassingly parallel grid workloads, requiring minimal code changes over the sequential baseline.
- Speedup is grid-size dependent: larger grids (600×600, 3.47×) consistently outperform smaller grids (200×200, 2.52×) at the same thread count.
- Parallel efficiency decreases with thread count but remains meaningful — 43% at 8 threads for the largest grid — suggesting the current VM has limited physical cores, causing HyperThreading contention at high thread counts.
- The granularity problem is evident for small grids at high thread counts, providing a practical lesson in choosing an appropriate level of parallelism for a given problem size.
- All timings are reproducible (fixed random seed srand(42)), and the parallel logic is provably race-condition-free due to read-only access to the current grid.

## 10. Future Work

- Refactor the grid to a flat 1D vector to improve cache locality and measure the performance delta.
- Extend to GPU acceleration using CUDA or OpenCL to leverage thousands of parallel cores.
- Implement an MPI-based distributed version for multi-node cluster execution.
- Add fire probability parameter p_fire to make the model stochastic and study statistical spread patterns.
- Profile with Intel VTune or perf to identify memory bandwidth vs. compute bottlenecks.